

IDS5981-0-Introduction...
PDF

IDS5981-1-Introduction...
PDF

these are some slides materials you can reference to.
read the slides materials carefully first. Then we are
going to implement the following 3 tasks one-by-one
carefully.

1.(10) Download data from
<https://coinmarketcap.com/historical/> by hand OR
through APIs from any crypto exchange you prefer (e.g.
coinbase, binance, etc.).

2.(10) Check and download data from
<https://www.blockchain.com/explorer/charts> , either by
hand or through API is fine.

3.(30) List the data you think that can be used to predict
BTC prices and explain the reasons behind each.
Quantitative explanations are preferred.

Thought for 8s >

Got it. Based on the lecture/assignment slides (they explicitly list Tasks 1–3, and also
motivate “intrinsic value” via hash rate + “circulating value” via transaction demand)

 IDS5981-1-Introduction to Block...

 IDS5981-1-Introduction to Block...

, here’s how to do **Tasks 1–3**
cleanly.

1) Download data from CoinMarketCap “Historical Snapshots”

Manual (fastest, no coding):

1. Go to coinmarketcap.com/historical/
2. Choose a snapshot date (they list dates like “2024...”, “2023...” etc.)

3. Open the snapshot → use the page's **Download/Export (if shown)**, or copy the table into CSV (browser "Save page" won't structure it well; best is copy table → paste into Google Sheets/Excel → save CSV).

Programmatic options (pick one):

- **CoinMarketCap API (official)**: best quality but requires an API key/subscription; you typically pull listings + market data by date/interval.
- **Exchange APIs (recommended for assignment practicality)**: if your goal is *tradeable* time series (BTC OHLCV), use:
 - **Binance**: kline/candlestick endpoint for BTCUSDT (e.g., 1m/5m/1h/1d)
 - **Coinbase**: candles endpoint for BTC-USD
 This avoids scraping CoinMarketCap tables and gives cleaner "price" targets for modeling.

(Why this matches the course framing: the course emphasizes quant workflow—data → backtest/validation → execution/monitoring  IDS5981-0-Introduction , and exchange OHLCV is the standard input for that.)

2) Download data from blockchain.com Explorer Charts

Manual:

1. Go to **blockchain.com/explorer/charts**
2. Pick a chart (e.g., "Hash Rate", "Transactions", "Difficulty", etc.)
3. Most charts let you export/download data (or you can read the values via the chart UI and export).

API (very convenient):

Blockchain.com charts commonly support a simple API format:

- JSON: `https://api.blockchain.info/charts/<chart-name>?timespan=all&format=json`
- CSV: `https://api.blockchain.info/charts/<chart-name>?timespan=all&format=csv`

Examples (you can swap `<chart-name>`):

- `market-price` (BTC price index)
- `hash-rate`
- `difficulty`

- `n-transactions`
- `mempool-count / mempool-size` (if available as chart names)

This lines up directly with the lecture emphasis on mining difficulty adjustments and hash rate dynamics  IDS5981-1-Introduction to Block...

3) Data that can help predict BTC price (with quantitative “why”)

Below are **predictor families** that are both *reasonable* and *measurable*. I'm using the lecture's framing that **hash rate relates to “intrinsic/cost”** and **transactions relate to “demand/circulating value”**  IDS5981-1-Introduction to Block...

A) Price/market microstructure (from exchanges)

1. Returns & momentum features

- Examples: past 1h/1d/1w returns, moving averages, trend strength.
- Quant idea: short-horizon autocorrelation often exists in liquid markets; momentum captures it.

2. Volatility / realized variance

- Example: rolling std of returns, Parkinson/Garman–Klass volatility using OHLC.
- Quant reason: volatility clusters; higher vol often predicts higher future vol and can affect risk premia (position sizing, liquidation risk).

3. Order book / liquidity

- Example: bid-ask spread, order book imbalance, depth at top N levels.
- Quant reason: price impact $\propto 1/\text{liquidity}$; widening spreads/depth drops can precede jumps (esp. around news/large flows).

4. Derivatives: funding rate, open interest, basis

- Funding rate (perp) $\approx$ crowd positioning; extreme positive funding can precede mean-reversion (crowded longs).
- Open interest spikes can signal leverage buildup $\rightarrow$ liquidation cascades.

B) On-chain “usage / demand”

5. Transaction count / active addresses

- Lecture-aligned: “total number of transactions reflects demand for BTC”

 IDS5981-1-Introduction to Block...

- Quant: growth rate of transactions/actives can proxy adoption; changes can lead price (especially at weekly/monthly horizons).

6. Transfer volume (on-chain), adjusted volume

- Quant: volume is a classic demand proxy; you'd typically log-transform and use $\Delta \log(\text{volume})$.

7. Mempool congestion + fee metrics

- Examples: mempool size, avg fee, fee to get into next block.
- Quant: congestion indicates urgency/demand for block space; rising fees can correlate with heightened activity and sometimes speculative bursts.

C) Mining / supply-side "cost & security"

8. Hash rate

- Lecture: hash rate $\leftrightarrow$ mining power $\leftrightarrow$ cost of minting/intrinsic value

 IDS5981-1-Introduction to Block...

- Quant reason: sustained hash-rate increases can indicate miner confidence/investment; sudden drops can signal stress.

9. Difficulty

- Difficulty adjusts ~every ~2016 blocks (~2 weeks) per lecture

 IDS5981-1-Introduction to Block...

- Quant: difficulty trend can lag hash-rate changes; the *gap* between hash rate changes and difficulty adjustment can matter for miner profitability.

10. Miner profitability proxies

- Example: $(\text{BTC price} \times \text{block subsidy} + \text{fees}) / \text{hash rate}$ (a rough "revenue per hash" proxy).
- Quant: when profitability compresses, miners may sell inventory (sell pressure) or shut off rigs (hash-rate drops).

D) Macro / cross-asset risk

11. USD liquidity / rates / DXY / equities

- Quant: BTC behaves like a high-beta risk asset at times; include SPX/Nasdaq returns, VIX, 10Y yields, DXY changes.

12. Stablecoin supply & flows (USDT/USDC)

- Quant: stablecoin net issuance can proxy "dry powder" entering crypto; large exchange inflows can precede buys/sells (direction depends on context).

E) Event & regime features (helpful in crypto)

13. Halving cycle / issuance schedule

- Quant: deterministic supply shock; usually modeled as regime dummies or time-since-halving.

14. Regime detection (chaotic vs predictable)

- The intro slides discuss distinguishing regimes and that strategy performance can differ across regimes [IDS5981-0-Introduction](#) .
- Quant: use volatility/entropy/market structure breaks to gate which predictors to trust.

If you want, we can implement these **one-by-one** exactly like the assignment workflow (data → validation → model) [IDS5981-0-Introduction](#) , starting with **Task 1**: choose either **Binance OHLCV** (cleanest) or **CoinMarketCap snapshots** (table-style cross-sectional).

Sources



good. now we are going to implement the optimal python code for the first task, using API to download the data. You can choose the most convenient API to use

Below is a **clean, "works-out-of-the-box"** Python solution for **Task 1** using the **Binance public REST API** (no API key needed). It downloads **BTCUSDT OHLCV** candles and saves to CSV.

Why Binance?

- Free, no auth for klines
- High quality OHLCV data
- Simple endpoint + stable

python

```

import time
import math
import requests
import pandas as pd
from datetime import datetime, timezone
from typing import Optional, List, Dict, Any

BINANCE_KLINES_URL = "https://api.binance.com/api/v3/klines"

# Binance interval options: '1m', '3m', '5m', '15m', '30m', '1h', '2h', '4h', '6h'
def interval_to_ms(interval: str) -> int:
    unit = interval[-1]
    num = int(interval[:-1])
    if unit == "m":
        return num * 60_000
    if unit == "h":
        return num * 3_600_000
    if unit == "d":
        return num * 86_400_000
    if unit == "w":
        return num * 7 * 86_400_000
    if unit == "M":
        # Binance "1M" is calendar month; for paging we don't rely on exact
        # We'll handle month paging via startTime increment using the last
        return -1
    raise ValueError(f"Unsupported interval: {interval}")

def to_ms(dt_str: str) -> int:
    """
    Parse ISO-like date strings into milliseconds since epoch (UTC).
    Examples:
        '2023-01-01'
        '2023-01-01 00:00:00'
        '2023-01-01T00:00:00'
    """
    s = dt_str.strip().replace("T", " ")
    if len(s) == 10:
        s += " 00:00:00"
    dt = datetime.strptime(s, "%Y-%m-%d %H:%M:%S").replace(tzinfo=timezone.utc)
    return int(dt.timestamp() * 1000)

```

```

def fetch_klines_page(symbol: str, interval: str, start_ms: int, end_ms: 0
    params: Dict[str, Any] = {
        "symbol": symbol,
        "interval": interval,
        "startTime": start_ms,
        "limit": limit,
    }
    if end_ms is not None:
        params["endTime"] = end_ms

    r = requests.get(BINANCE_KLINES_URL, params=params, timeout=20)
    r.raise_for_status()
    return r.json()

def download_binance_ohlcv(
    symbol: str = "BTCUSDT",
    interval: str = "1d",
    start: str = "2017-01-01",
    end: Optional[str] = None,
    out_csv: str = "btc_ohlcv_binance.csv",
) -> pd.DataFrame:
    """
    Downloads Binance OHLCV klines for [start, end) and saves to CSV.

    - Handles Binance 1000-candle/page limit by paging with startTime.
    - Uses last candle open time to advance safely.
    """
    start_ms = to_ms(start)
    end_ms = to_ms(end) if end else None

    all_rows: List[List[Any]] = []
    page = 0

    while True:
        page += 1
        rows = fetch_klines_page(symbol, interval, start_ms, end_ms, limit
        if not rows:
            break

```

```
all_rows.extend(rows)

# Advance start_ms to (last_open_time + 1ms) to avoid duplicates
last_open_time = rows[-1][0]
next_start = last_open_time + 1

# Stop if we are not progressing (safety guard)
if next_start <= start_ms:
    break

start_ms = next_start

# If fewer than limit returned, we've reached the end
if len(rows) < 1000:
    break

# Be polite to API
time.sleep(0.25)

if not all_rows:
    raise RuntimeError("No data returned. Check symbol/interval/date r

cols = [
    "open_time_ms", "open", "high", "low", "close", "volume",
    "close_time_ms", "quote_asset_volume", "num_trades",
    "taker_buy_base_volume", "taker_buy_quote_volume", "ignore"
]
df = pd.DataFrame(all_rows, columns=cols)

# Convert types
float_cols = ["open", "high", "low", "close", "volume", "quote_asset_v
               "taker_buy_base_volume", "taker_buy_quote_volume"]
int_cols = ["open_time_ms", "close_time_ms", "num_trades"]

for c in float_cols:
    df[c] = df[c].astype(float)
for c in int_cols:
    df[c] = df[c].astype(int)

# Add readable timestamps
```

```

df["open_time"] = pd.to_datetime(df["open_time_ms"], unit="ms", utc=True)
df["close_time"] = pd.to_datetime(df["close_time_ms"], unit="ms", utc=True)

# Keep neat, common OHLCV columns first
df = df[[
    "open_time", "open", "high", "low", "close", "volume",
    "quote_asset_volume", "num_trades",
    "taker_buy_base_volume", "taker_buy_quote_volume",
    "close_time"
]]

df.to_csv(out_csv, index=False)
return df

if __name__ == "__main__":
    # Example: daily candles from 2017 to now
    df = download_binance_ohlcv(
        symbol="BTCUSDT",
        interval="1d",
        start="2017-01-01",
        end=None,
        out_csv="btc_ohlcv_binance_1d.csv",
    )
    print(df.head())
    print(f"\nSaved {len(df)} rows to btc_ohlcv_binance_1d.csv")

```

What you'll get

A CSV with columns like:

- open_time, open, high, low, close, volume
- plus trades + taker buy volumes (useful later)

If you tell me what granularity you want (**1m / 1h / 1d**) and what time range (e.g., **last 5 years**), I'll set the exact parameters and make it "assignment-ready" (including basic checks + plotting).



first of all, we got the following error. Second, I want to make the time, coins, and etc to be python args that are able to be customized in the input

```
(stripe-interview) zanwenfu@Mac assignment1 % python lec1.py
```

Traceback (most recent call last):

```
File "/Users/zanwenfu/Documents/Crypto
Trading/assignments/assignment1/lec1.py", line 137, in
<module>
```

```
    df = download_binance_ohlcv(
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/Users/zanwenfu/Documents/Crypto
Trading/assignments/assignment1/lec1.py", line 77, in
download_binance_ohlcv
```

```
    rows = fetch_klines_page(symbol, interval, start_ms,
end_ms, limit=1000)
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/Users/zanwenfu/Documents/Crypto
Trading/assignments/assignment1/lec1.py", line 53, in
fetch_klines_page
```

```
    r.raise_for_status()
```

```
File "/Users/zanwenfu/miniforge3/envs/stripe-
interview/lib/python3.11/site-
packages/requests/models.py", line 1021, in
raise_for_status
```

```
    raise HTTPError(http_error_msg, response=self)
requests.exceptions.HTTPError: 451 Client Error: for url:
https://api.binance.com/api/v3/klines?
symbol=BTCUSDT&interval=1d&startTime=1483228800
000&limit=1000
```

Thought for 7s >

Yep — that **451** is the key: **Binance's public API is geo-restricted in some regions (commonly the US), so the server returns "Unavailable For Legal Reasons (451)".** Nothing wrong with your code logic; it's the endpoint access.

So the "optimal" fix is: **switch to a public API that works reliably from the US**, and add **CLI args** for coin / time / interval / output.

Below is a drop-in script using **Kraken public API** (no key, usually accessible in the US). It supports:

- `--pair` (e.g. XBTUSD , XBTUSDT , ETHUSD)
- `--interval` minutes (1,5,15,60,240,1440,...)
- `--start` , `--end` (YYYY-MM-DD or with time)
- `--out` CSV filename

python

```
import argparse
import time
import requests
import pandas as pd
from datetime import datetime, timezone
from typing import Optional, Dict, Any, List

KRAKEN_OHLC_URL = "https://api.kraken.com/0/public/OHLC"

def parse_dt_to_unix(s: str) -> int:
    """
    Accepts:
        '2020-01-01'
        '2020-01-01 13:30:00'
        '2020-01-01T13:30:00'
    Returns unix seconds (UTC).
    """
    s = s.strip().replace("T", " ")
    if len(s) == 10:
        s += " 00:00:00"
    dt = datetime.strptime(s, "%Y-%m-%d %H:%M:%S").replace(tzinfo=timezone
```

```

    return int(dt.timestamp())

def fetch_kraken_ohlc_page(pair: str, interval: int, since: int) -> Dict[s
    params = {"pair": pair, "interval": interval, "since": since}
    r = requests.get(KRAKEN_OHLC_URL, params=params, timeout=20)
    r.raise_for_status()
    data = r.json()
    if data.get("error"):
        raise RuntimeError(f"Kraken API error: {data['error']}")
    return data["result"]

def download_kraken_ohlc(
    pair: str,
    interval: int,
    start: str,
    end: Optional[str] = None,
    out_csv: str = "ohlc_kraken.csv",
    sleep_s: float = 0.25,
) -> pd.DataFrame:
    """
    Downloads OHLCV candles from Kraken public API and saves to CSV.

    Kraken returns:
        [time, open, high, low, close, vwap, volume, count]
    Pagination:
        use returned 'last' as next since cursor.
    """
    start_unix = parse_dt_to_unix(start)
    end_unix = parse_dt_to_unix(end) if end else None

    all_rows: List[List[Any]] = []
    since = start_unix
    seen_last = None

    while True:
        result = fetch_kraken_ohlc_page(pair=pair, interval=interval, sinc

        # The OHLC data is under a dynamic key (pair name), and 'last' cur
        last_cursor = int(result["last"])
        pair_keys = [k for k in result.keys() if k != "last"]

```

```
if not pair_keys:
    break
ohlcv_key = pair_keys[0]
rows = result[ohlcv_key]

if not rows:
    break

# Filter rows to [start, end) if end provided
if end_unix is not None:
    rows = [row for row in rows if int(row[0]) < end_unix]

all_rows.extend(rows)

# Stop conditions
if end_unix is not None and all_rows and int(all_rows[-1][0]) >= e:
    break

# If cursor doesn't move, stop to avoid infinite loop
if seen_last is not None and last_cursor == seen_last:
    break
seen_last = last_cursor

# Advance
since = last_cursor
time.sleep(sleep_s)

# If we got filtered to empty due to end_unix, we can stop
if end_unix is not None and not rows:
    break

if not all_rows:
    raise RuntimeError("No OHLC data returned. Check pair/interval/dat

df = pd.DataFrame(
    all_rows,
    columns=["time_unix", "open", "high", "low", "close", "vwap", "vol
)

# types
```

```

df["time_unix"] = df["time_unix"].astype(int)
for c in ["open", "high", "low", "close", "vwap", "volume"]:
    df[c] = df[c].astype(float)
df["count"] = df["count"].astype(int)

df["time"] = pd.to_datetime(df["time_unix"], unit="s", utc=True)

# neat column order
df = df[["time", "open", "high", "low", "close", "volume", "vwap", "co

df.to_csv(out_csv, index=False)
return df

def main():
    parser = argparse.ArgumentParser(description="Download OHLCV data via
    parser.add_argument("--pair", type=str, default="XBTUSD",
                        help="Kraken trading pair, e.g., XBTUSD, XBTUSD,
    parser.add_argument("--interval", type=int, default=1440,
                        help="Candle interval in minutes. Common: 1,5,15,6
    parser.add_argument("--start", type=str, default="2017-01-01",
                        help="Start datetime (UTC): 'YYYY-MM-DD' or 'YYYY-
    parser.add_argument("--end", type=str, default=None,
                        help="End datetime (UTC): same format as start. If
    parser.add_argument("--out", type=str, default="ohlcv_kraken.csv",
                        help="Output CSV filename")
    args = parser.parse_args()

    df = download_kraken_ohlcv(
        pair=args.pair,
        interval=args.interval,
        start=args.start,
        end=args.end,
        out_csv=args.out,
    )
    print(df.head())
    print(f"\nSaved {len(df)} rows to {args.out}")

if __name__ == "__main__":
    main()

```

Example runs

Daily BTC-USD since 2017:

```
bash
```

```
python lec1.py --pair XBTUSD --interval 1440 --start 2017-01-01 --out btc_
```

Hourly BTC-USDT for 2023:

 Copy code

```
bash
```

```
python lec1.py --pair XBTUSDT --interval 60 --start 2023-01-01 --end 2024-
```

 Copy code

If you want, I can also add a **friendlier "coin" input** like `--base BTC --quote USD` and auto-map it to Kraken's pair symbols (XBT vs BTC, etc.).

     ...

Good. Now we are moving on to implement the second task "Check and download data from <https://www.blockchain.com/explorer/charts> , either by hand or through API is fine.". Again use APIs with configurable input args

Here's an "assignment-ready" **Task 2** script that downloads **any Blockchain.com Explorer chart** via their public charts API, with **configurable CLI args** (chart name, timespan, start/end, format, output file).

It supports:

- `--chart` (e.g. `hash-rate` , `difficulty` , `n-transactions` , `market-price`)
- `--timespan` (e.g. `all` , `5years` , `365days` , `30days`)
- `--start` / `--end` (optional; if provided we'll filter locally)
- `--format` (`json` or `csv`)
- `--out` output filename

python

```
import argparse
import io
import requests
import pandas as pd
from datetime import datetime, timezone
from typing import Optional

BASE_URL = "https://api.blockchain.info/charts"

def parse_dt_utc(s: str) -> datetime:
    """
    Accepts:
        '2020-01-01'
        '2020-01-01 13:30:00'
        '2020-01-01T13:30:00'
    Returns timezone-aware UTC datetime.
    """
    s = s.strip().replace("T", " ")
    if len(s) == 10:
        s += " 00:00:00"
    dt = datetime.strptime(s, "%Y-%m-%d %H:%M:%S")
    return dt.replace(tzinfo=timezone.utc)

def fetch_chart_raw(chart: str, timespan: str, fmt: str) -> str:
    """
    Fetch raw chart data from Blockchain.com charts endpoint.
    fmt: 'json' or 'csv'
    """
    url = f"{BASE_URL}/{chart}"
    params = {
        "timespan": timespan,
        "format": fmt,
    }
    r = requests.get(url, params=params, timeout=20)
    r.raise_for_status()
    return r.text
```

```

def parse_chart_json_to_df(text: str) -> pd.DataFrame:
    """
    Blockchain charts JSON looks like:
        { "status": "...", "name": "...", "values": [{"x": 1234567890, "y":
    where x is unix seconds.
    """
    data = requests.models.complexjson.loads(text)
    if "values" not in data:
        raise RuntimeError(f"Unexpected JSON format. Keys: {list(data.keys)}")
    df = pd.DataFrame(data["values"])
    if df.empty:
        return df
    df["time"] = pd.to_datetime(df["x"], unit="s", utc=True)
    df = df.rename(columns={"y": "value"})[["time", "value"]]
    return df


def parse_chart_csv_to_df(text: str) -> pd.DataFrame:
    """
    CSV usually like:
        Timestamp,Value
        2012-01-01, ...
    We try to parse robustly.
    """
    df = pd.read_csv(io.StringIO(text))
    # Normalize column names
    cols_lower = {c: c.strip().lower() for c in df.columns}
    df = df.rename(columns=cols_lower)

    # Common possibilities: 'timestamp' or 'date' + 'value'
    time_col = None
    for c in ["timestamp", "date", "time"]:
        if c in df.columns:
            time_col = c
            break
    if time_col is None or "value" not in df.columns:
        raise RuntimeError(f"Unexpected CSV columns: {df.columns.tolist()}")

    df["time"] = pd.to_datetime(df[time_col], utc=True, errors="coerce")
    df = df.dropna(subset=["time"]).copy()
    df["value"] = pd.to_numeric(df["value"], errors="coerce")

```

```

df = df.dropna(subset=["value"]).copy()
df = df[["time", "value"]]
return df

def filter_df_by_range(df: pd.DataFrame, start: Optional[str], end: Option
    if df.empty:
        return df
    if start:
        sdt = parse_dt_utc(start)
        df = df[df["time"] >= sdt]
    if end:
        edt = parse_dt_utc(end)
        df = df[df["time"] < edt]
    return df

def download_blockchain_chart(
    chart: str,
    timespan: str,
    fmt: str,
    start: Optional[str],
    end: Optional[str],
    out: str,
) -> pd.DataFrame:
    raw = fetch_chart_raw(chart=chart, timespan=timespan, fmt=fmt)

    # Parse
    if fmt == "json":
        df = parse_chart_json_to_df(raw)
    elif fmt == "csv":
        df = parse_chart_csv_to_df(raw)
    else:
        raise ValueError("fmt must be 'json' or 'csv'")

    # Optional local filtering
    df = filter_df_by_range(df, start, end)

    # Save
    df.to_csv(out, index=False)
    return df

```

```
def main():
    parser = argparse.ArgumentParser(
        description="Download Blockchain.com Explorer chart data via API."
    )
    parser.add_argument(
        "--chart",
        type=str,
        required=True,
        help="Chart name from https://www.blockchain.com/explorer/charts (
    )
    parser.add_argument(
        "--timespan",
        type=str,
        default="all",
        help="Timespan string (e.g., all, 5years, 365days, 30days)",
    )
    parser.add_argument(
        "--format",
        type=str,
        choices=["json", "csv"],
        default="json",
        help="API response format to request",
    )
    parser.add_argument(
        "--start",
        type=str,
        default=None,
        help="Optional start datetime (UTC): 'YYYY-MM-DD' or 'YYYY-MM-DD H
    )
    parser.add_argument(
        "--end",
        type=str,
        default=None,
        help="Optional end datetime (UTC): same format as start",
    )
    parser.add_argument(
        "--out",
        type=str,
        default=None,
        help="Output CSV filename (default auto-named).",
```

```

    )
    args = parser.parse_args()

    out = args.out or f"blockchain_{args.chart}_{args.timespan}.csv"
    df = download_blockchain_chart(
        chart=args.chart,
        timespan=args.timespan,
        fmt=args.format,
        start=args.start,
        end=args.end,
        out=out,
    )
    print(df.head())
    print(f"\nSaved {len(df)} rows to {out}")

if __name__ == "__main__":
    main()

```

Example runs

Download **hash rate** (all time):

```
bash
```

```
python task2_blockchain.py --chart hash-rate --timespan all --format json
```

Download **transactions** for the last year:

 Copy code

```
bash
```

```
python task2_blockchain.py --chart n-transactions --timespan 365days --for
```

Download **difficulty** and filter locally to 2023:

 Copy code

```
bash
```

```
python task2_blockchain.py --chart difficulty --timespan 5years --start 20
```

If you want, I can also add a `--list-charts` option (scrape chart names once) so you can autocomplete valid chart IDs without guessing.

 Copy code

Good. Now the third task is a write-up question "List the data you think that can be used to predict BTC prices and explain the reasons behind each. Quantitative explanations are preferred.". Carefully review all the typical and common sectors that are considered in BTC predictions, and we are going to give straightforward reasoning offering key and instructional insights. You can be comprehensive regarding the reasoning, and choose the top 6 sectors. Avoid AI tone and be natural.

Here are the **top 6 "sectors" of data** people actually use when trying to predict BTC, with **straightforward quantitative intuition** for why each can matter. (None of these is magic alone — most of the value is in combining them + picking the right horizon.)

1) Price + market microstructure (spot trading data)

What to use

- OHLCV returns (1h/1d/1w), momentum (MA cross, trend), realized volatility
- Volume spikes, liquidity (spread, depth), order-book imbalance (if you have it)

Why it matters (quant intuition)

- **Volatility clusters:** high vol today → higher expected vol tomorrow (risk regimes). That changes how prices move and how quickly they mean-revert.
 - **Liquidity is price impact:** when depth drops / spreads widen, the same net buying can move price more (bigger jumps).
 - **Momentum often shows up at crypto horizons** (hours → days) because flows don't instantly equilibrate. Even a simple feature like past 7-day return can carry signal in trending regimes.
-

2) Derivatives positioning (perps + futures)

What to use

- Funding rate (perpetual swaps), open interest (OI), futures basis, liquidations, long/short ratio (if available)

Why it matters (quant intuition)

- Funding is basically **the cost of being levered long/short**. When funding is extremely positive, longs are crowded and paying shorts — that tends to mean **fragile upside** and higher probability of pullbacks.
 - **OI rising + price rising** often means leverage is being added. That can amplify moves, but also increases **liquidation cascade risk** (forced selling when price drops through key levels).
 - Liquidation data is literally a “forced flow” proxy; forced flow is one of the few things that can move price fast.
-

3) On-chain usage / demand (blockchain activity)

What to use (from blockchain.com charts)

- Transactions count, active addresses (if available), transfer volume, mempool size, fee rates, average fee, confirmation delay

Why it matters (quant intuition)

- On-chain activity is a rough proxy for **real usage demand** (payments, exchange settlement, arbitrage, etc.).
 - Mempool + fees represent **demand for block space**. Spikes in fees often coincide with high activity periods and sometimes speculative bursts.
 - These features tend to be more useful on **multi-day to multi-week horizons** than on intraday trading.
-

4) Mining + supply-side economics (cost & security)

What to use

- Hash rate, difficulty, block production / time between blocks
- “Miner revenue” proxies: (subsidy + fees) × price, revenue-per-hash (roughly revenue/hashrate)

Why it matters (quant intuition)

- Hash rate/difficulty reflect the **capital committed to securing BTC**. Big drops can signal miner stress (power cost, regulation shocks, hardware unplugging).
 - Miner stress can turn into **sell pressure**. If profitability collapses, miners may sell reserves to cover OPEX — that's a real supply flow hitting the market.
 - Difficulty adjusts with lag, so the "gap" between hash rate changes and difficulty can matter for profitability dynamics.
-

5) Macro + cross-asset risk (the "global regime")

What to use

- Equity indices (Nasdaq/S&P returns), VIX, USD strength (DXY), rates (2Y/10Y), credit spreads
- Sometimes gold, oil, and global liquidity proxies

Why it matters (quant intuition)

- BTC often trades like a **high-beta risk asset** in many regimes. When real yields rise or risk-off spikes, BTC tends to get hit alongside other risky assets.
 - Macro variables help you detect whether you're in a "risk-on momentum" regime vs "risk-off deleveraging" regime — which changes how well crypto-native signals work.
-

6) Flows + "capital on-ramps" (stablecoins, exchange flows, ETFs)

What to use

- Stablecoin supply changes (USDT/USDC), large inflow/outflow events
- Exchange inflows/outflows (BTC to exchanges vs leaving exchanges)
- ETF flows (if you include traditional-market access)

Why it matters (quant intuition)

- Price moves when **net buying pressure** shows up. Flows are the most direct proxy for that.
- Rising stablecoin supply can mean more "dry powder" entering crypto markets.
- Large exchange inflows of BTC can be bearish short-term (coins moved to sell), while large outflows can signal accumulation — not perfect, but often informative when combined with trend/liquidity.

Practical “instructional” takeaway

If you’re building a predictor:

- **Intraday (minutes–hours):** microstructure + derivatives + liquidations dominate.
- **Swing (days–weeks):** momentum + on-chain demand + flows tend to matter more.
- **Regime filter** (vol + macro) is huge: many signals work only in certain regimes.

If you want, I can help you turn this into a clean 1–2 page write-up format for the assignment (with a short intro + bulletized “data → mechanism → how to quantify” for each sector).



alright now we are going to turn this into professional essay or report format, offering the important and detailed reasoning, with absolutely quantitative explanations (prioritized).

Below is a **professional, report-style write-up** suitable for a quantitative finance / crypto markets assignment. I keep the tone academic and practical, and I **prioritize quantitative mechanisms** over narrative claims.

 Copy code

Data Sources for Predicting Bitcoin (BTC) Prices: A Quantitative Perspective

Introduction

Bitcoin (BTC) is a hybrid asset that simultaneously exhibits properties of a speculative financial instrument, a payment network, and a computationally secured commodity. As a result, no single category of data is sufficient to explain or predict BTC price movements across all horizons. Instead, predictive modeling typically relies on combining signals from multiple sectors, each capturing a different economic mechanism that influences price formation.

This report identifies **six major data sectors** commonly used in BTC price prediction and provides **quantitative reasoning** for why each sector may contain predictive information. The emphasis is on mechanisms that directly affect **demand, supply, leverage, liquidity, or risk regimes**, which are the fundamental drivers of price changes in liquid markets.

1. Market Price and Microstructure Data

Data Examples

- OHLCV (Open, High, Low, Close, Volume)
- Returns and momentum (e.g., rolling returns, moving averages)
- Realized volatility
- Bid–ask spread and liquidity measures (if available)

Quantitative Rationale

Market price data captures **realized trading outcomes** and embeds the aggregate behavior of all market participants. Two well-documented quantitative properties motivate its use:

1. Volatility clustering

Empirically, the conditional variance of BTC returns exhibits persistence:

$$\mathbb{E}[\sigma_{t+1}^2 \mid \sigma_t^2] > \mathbb{E}[\sigma^2]$$

This implies that high-volatility regimes tend to persist, which affects both risk-adjusted returns and the probability of extreme price moves.

2. Trend persistence and momentum

Over short to medium horizons (hours to days), BTC often shows positive autocorrelation in returns:

$$\text{corr}(r_t, r_{t-k}) > 0 \quad \text{for small } k$$

This can arise from slow-moving capital, delayed information diffusion, or systematic trading strategies.

Volume and liquidity metrics matter because **price impact is inversely proportional to liquidity**. For a given net order flow Q , the expected price change satisfies:

$$\Delta P \propto \frac{Q}{\text{Market Depth}}$$

Thus, identical demand shocks can lead to very different price outcomes depending on liquidity conditions.

2. Derivatives and Leverage Positioning

Data Examples

- Perpetual swap funding rates
- Open interest (OI)
- Futures basis
- Liquidation volumes

Quantitative Rationale

Crypto markets are heavily influenced by leverage. Derivatives data captures **latent positioning** that is not visible in spot prices alone.

- **Funding rates** approximate the cost of leverage:

$$\text{Funding} \approx \text{Price}_{\text{perp}} - \text{Price}_{\text{spot}}$$

Persistently positive funding indicates crowded long positions. In such conditions, the market becomes fragile: a small negative price shock can trigger forced deleveraging.

- **Open interest growth** measures leverage accumulation. When OI rises faster than spot volume, it often signals increased liquidation risk rather than organic demand.
- **Liquidations** represent non-discretionary trades. Forced selling creates negative convexity: once a threshold is crossed, selling pressure accelerates mechanically.

Empirically, large liquidation events are associated with high short-term volatility and directional continuation immediately after the event.

3. On-Chain Usage and Demand Metrics

Data Examples

- Number of transactions

- Transfer volume
- Mempool size and congestion
- Average transaction fees

Quantitative Rationale

On-chain metrics approximate **fundamental network usage**, which acts as a demand-side signal.

- Transaction count and volume proxy the intensity of economic activity on the network.
- Mempool congestion and fees reflect demand for scarce block space.

From a supply–demand perspective, block space is a fixed-capacity good. When demand exceeds capacity, fees rise:

$$\text{Fee} \uparrow \Rightarrow \text{Block Space Demand} > \text{Supply}$$

Fee spikes often coincide with periods of heightened activity, including speculative phases. While noisy at short horizons, sustained increases in on-chain activity can be informative over **weekly to monthly horizons**, especially when combined with price trends.

4. Mining and Supply-Side Economics

Data Examples

- Hash rate
- Mining difficulty
- Block production rate
- Estimated miner revenue

Quantitative Rationale

Bitcoin supply is secured by computational work. Mining-related data captures the **cost structure and security commitment** of the network.

- Hash rate reflects total computational investment. Sustained increases imply capital inflow into mining infrastructure.
- Difficulty adjusts with lag, creating temporary profitability shocks.

A useful proxy is **revenue per unit of hash**:

$$\text{Revenue per Hash} \approx \frac{(\text{Block Reward} + \text{Fees}) \times \text{BTC Price}}{\text{Hash Rate}}$$

When revenue per hash collapses, miners may:

1. Shut down machines (hash rate drops), or
2. Sell BTC reserves to cover operating costs (increasing sell pressure).

Thus, mining data links directly to **structural supply flows**, not just sentiment.

5. Macro and Cross-Asset Risk Indicators

Data Examples

- Equity indices (e.g., Nasdaq, S&P 500)
- Volatility indices (VIX)
- Interest rates and yield curves
- USD strength (DXY)

Quantitative Rationale

Despite its unique properties, BTC often behaves as a **high-beta risk asset** during certain regimes.

Empirically:

$$\beta_{\text{BTC, equities}} > 0 \quad \text{in risk-on regimes}$$

When real yields rise or global volatility spikes, capital tends to move away from speculative assets, including BTC. Macro variables are therefore useful as **regime filters**: they help determine whether crypto-native signals (e.g., momentum or on-chain metrics) are likely to work or fail.

6. Capital Flows and Market Access Channels

Data Examples

- Stablecoin supply changes (USDT, USDC)
- Exchange inflows/outflows

- ETF inflows/outflows (where applicable)

Quantitative Rationale

Ultimately, price changes require **net capital flows**.

- Stablecoins act as settlement currency in crypto markets. Net issuance can proxy available buying power.
- Exchange inflows of BTC often precede selling; outflows may indicate accumulation or cold storage.
- ETF flows represent access by traditional capital pools and can dominate marginal demand during certain periods.

Flow-based variables are among the most direct predictors because they measure **actual or imminent transactions**, not inferred intent.

Conclusion

BTC price dynamics are driven by a combination of **market mechanics, leverage, network usage, supply-side economics, macro regimes, and capital flows**. Each data sector captures a distinct economic mechanism, and none is sufficient in isolation.

From a quantitative perspective:

- Short horizons are dominated by **microstructure and leverage**.
- Medium horizons benefit from **momentum, flows, and on-chain demand**.
- Longer horizons require **macro and supply-side context**.

Effective BTC prediction therefore depends less on finding a single “best” indicator and more on **integrating complementary signals while accounting for regime changes**.

If you want, I can now:

- Convert this into a **LaTeX-ready academic report**, or
- Add **equations + references** to make it publication-style, or
- Help you explicitly link this write-up to the **datasets you downloaded in Tasks 1 and 2** (very good for grading).



Now as a senior crypto trader or a senior blockchain professor, what do you think of this write-up? Any instructional feedback or advice to improve?

As a **senior crypto trader** *and* a **blockchain professor**, I'd say this is a **strong, above-average write-up** for a quantitative crypto course. It's clear, structured, and—most importantly—you focused on **mechanisms**, not vibes. That already puts it ahead of most submissions.

That said, if the goal is to push this from **"very good"** → **"excellent / top-tier"**, here is **honest, instructional feedback**.

Overall Assessment

What you did very well

- You correctly framed BTC as a **hybrid asset** (commodity + network + speculative instrument).
- Each data sector is tied to a **price formation mechanism** (flow, leverage, cost, risk regime).
- You used equations *only where they add clarity* (good judgment).
- You implicitly distinguished **time horizons**, which many people forget.

This already reads like something written by someone who has actually **looked at BTC data**, not just read blog posts.

How to Improve (Concrete, High-Value Suggestions)

1. Make the prediction horizon explicit (this is the biggest upgrade)

Right now, horizons are implied but not formalized.

What's missing

- A clear mapping:

- Which signals work at which time scale?
- Which ones break down outside their horizon?

How to fix

Add a short table or paragraph like:

Market microstructure and derivatives variables primarily affect **intra-day to multi-day** price dynamics due to leverage and liquidity effects.

On-chain usage and mining economics are more informative at **weekly to monthly** horizons, where fundamental demand and supply adjustments can materialize.

This shows **modeling maturity**, not just knowledge.

2. Be explicit about causality vs correlation

You do a good job implicitly, but graders (and professors) care about this distinction.

What you can tighten

- Some sections may read as "X predicts Y" without clarifying *why prediction might fail*.

How to fix

Add cautionary language like:

- "These variables are **conditionally predictive**, often only under specific regimes."
- "On-chain metrics are not causal drivers but **slow-moving demand proxies**."

This demonstrates **scientific restraint**, which professors respect.

3. Strengthen the mining section with one sharper insight

The mining section is correct but slightly "textbook".

Upgrade it with one professional-level point

- Mention **hash rate is largely exogenous at short horizons**.
- Emphasize **miner inventory behavior**, not just cost.

Example improvement:

While hash rate reflects long-term security investment, short-term price effects arise primarily through miner balance sheet constraints, which determine whether BTC is sold or retained during profitability shocks.

This shows you understand **who actually hits the sell button**.

4. Add a brief “failure modes” paragraph (very strong signal)

Top-tier write-ups show **when models fail**, not just when they work.

You could add a short section:

Many BTC predictors fail during regime transitions, such as post-halving periods, regulatory shocks, or sudden liquidity withdrawals. Models trained on stable regimes often extrapolate incorrectly when leverage, macro conditions, or market access changes abruptly.

This demonstrates **trading realism**, not academic idealism.

5. Slightly reduce symmetry across sections

Every section currently has a similar structure (data → rationale → equation). That’s clean, but a bit “too perfect”.

Why this matters

- Real thinking is slightly asymmetric.
- Some factors are weaker, noisier, or secondary.

Suggestion

Explicitly rank or qualify importance:

- “Among on-chain metrics, transaction count is noisier than fee pressure.”
- “Macro variables dominate during global stress but are nearly irrelevant during isolated crypto bull runs.”

That shows **judgment**, not just completeness.

Final Verdict

Grade (if I were teaching): A / A+

Trader's verdict:

You're thinking in the *right mental model* — flows, leverage, cost, regimes. That's how real BTC desks think.

What would push this into "publication-level"

- Explicit horizon mapping
- Clear causality disclaimers
- One or two sharp, non-obvious insights (miner inventory, regime failure)

If you want, next we can:

- Add a **1-page executive summary** (very impressive for grading), or
- Turn this into a **model design blueprint** (features → horizon → expected failure modes), or
- Align this directly with a **backtesting plan** (what to test first, what not to bother testing).

Just tell me the direction.



Good. now review the feedback and the write-up, we are going to refine and improve the write-up to make it the best and most optimal version ever. be quantitative and comprehensive, and avoid AI tone

Data for Predicting Bitcoin (BTC) Prices: A Quantitative, Mechanism-Driven View

Introduction

BTC is not "one thing." It trades as (i) a **high-beta risk asset** in many macro regimes, (ii) a **leveraged derivatives-driven market** intraday, and (iii) a **scarce network good** over

longer horizons. Because the dominant mechanism changes with time scale, the most useful predictors are the ones that map cleanly to **order flow, leverage, liquidity, supply pressure, and regime shifts**.

A practical way to structure prediction is by **horizon**:

- **Intraday (minutes → hours)**: microstructure + derivatives positioning dominate; forced flows matter.
- **Swing (days → weeks)**: trend + flows dominate; liquidation risk and macro conditions modulate.
- **Structural (weeks → months)**: macro liquidity + adoption/usage proxies + supply-side constraints dominate.

Below are six data sectors that cover the common “toolbox” used in BTC prediction, with quantitative reasoning and clear failure modes.

1) Spot Price Action & Market Microstructure (the “what the market actually did”)

Typical data

- OHLCV (spot): returns $r_t = \ln(P_t/P_{t-1})$, volume, high-low range
- Realized volatility: $\sigma_t = \sqrt{\sum_{i=1}^n r_{t,i}^2}$
- Liquidity proxies: bid–ask spread, market depth, order-book imbalance (if available)

Why it can predict

1. Volatility clustering / regime persistence

BTC’s conditional variance is persistent. A simple model like:

$$\sigma_{t+1}^2 \approx \alpha + \beta \sigma_t^2 \quad (\beta > 0)$$

often beats a constant-vol assumption. This matters because many trading edges are *regime-conditional* (a mean-reversion strategy can look great in low vol and die in high vol).

2. Trend/momentum in flow-driven regimes

Over hours to days, price often trends because capital and information are slow to fully equilibrate. A clean quantitative feature is the k -day return:

$$m_t = \ln(P_t/P_{t-k})$$

Momentum isn't "always true," but when it exists it's typically strongest when liquidity is improving and leverage isn't excessively crowded.

3. Liquidity controls price impact

For a net aggressive flow Q_t , a basic microstructure approximation is:

$$\Delta P_t \propto \frac{Q_t}{\text{Depth}_t}$$

So the same buying pressure moves price more when order books are thin. This is why spread/depth variables can forecast *the distribution* of future returns (tail risk), even when direction is unclear.

Failure modes

- Microstructure signals decay quickly and can invert during regime transitions (e.g., news shocks).
- Momentum breaks in choppy, mean-reverting regimes or when leverage is overly one-sided.

2) Derivatives Positioning & Leverage (the "hidden state" behind spot)

Typical data

- Perpetual funding rate f_t
- Open interest (OI)
- Futures basis / premium
- Liquidations (size, direction)

Why it can predict

1. Funding rate as crowding / carry pressure

Funding is the recurring payment between longs and shorts. Extreme positive funding implies crowded longs; extreme negative implies crowded shorts. Crowding matters because it increases the probability of *nonlinear forced flow*:

- crowded longs + small down move → margin calls → **forced selling** → larger down move

2. Open interest growth = leverage buildup

Rising OI, especially when spot volume and liquidity do not rise proportionally, often signals leverage accumulation. Leverage makes the market fragile: the payoff is convex in the wrong direction because liquidations are threshold-driven.

3. Liquidations are non-discretionary order flow

Liquidations are literally "must trade now." Their predictive value is strongest over short horizons because they create autocorrelated flow:

- initial liquidation wave → price overshoot → follow-through or snapback depending on liquidity and whether the market is one-sided.

Failure modes

- Funding can stay extreme for long periods in persistent bull/bear regimes; using it as an immediate reversal signal without regime context often loses.
 - OI is ambiguous: it can reflect hedging, not directional leverage.
-

3) Flow & Market Access Channels (the closest thing to "demand" you can measure)

Typical data

- Exchange net flows (BTC to exchanges vs off exchanges)
- Stablecoin supply changes (USDT/USDC net issuance) and exchange stablecoin balances
- ETF flows (when applicable)

Why it can predict

Prices move when **net buying pressure** exceeds **net selling pressure**. Flows are direct proxies for that.

1. Exchange inflows/outflows as intent proxies

- BTC inflow to exchanges can precede selling (increased immediately-available supply).
- BTC outflow can imply accumulation/custody (reduced immediate supply).

This is noisy but becomes useful when combined with trend and liquidity: e.g., large inflows during weakening liquidity can amplify drawdowns.

2. Stablecoin net issuance as "dry powder"

Stablecoins act as settlement currency in crypto. Net issuance can proxy incremental purchasing capacity. A simple signal is $\Delta \ln(\text{Stablecoin Supply})$, often more useful on **days–weeks** horizons.

3. ETF flows as marginal institutional demand

ETF flow series can dominate marginal demand in certain regimes. Even if your model is crypto-native, ETF flows can explain price moves that on-chain metrics don't.

Failure modes

- Flow data can be gamed or misinterpreted (internal exchange movements).
- Stablecoin issuance can lag price (reactive demand), so directionality must be tested.

4) On-Chain Usage & Congestion (network demand for block space)

Typical data (Blockchain.com charts)

- Transaction count
- Transfer volume (preferably adjusted)
- Mempool size / congestion
- Fee level (average fee, fee rate)

Why it can predict

On-chain variables are best treated as **slow-moving demand proxies**.

1. Block space is a capacity-constrained good

When demand exceeds block capacity, fees rise:

$$\text{Fees}_t \uparrow \Rightarrow \text{Demand for inclusion} > \text{Supply of block space}$$

Sustained fee pressure signals elevated economic activity and often coincides with speculative intensity.

2. Transactions/volume as adoption/activity proxies

Growth rates like $\Delta \ln(\text{Tx Count})$ or $\Delta \ln(\text{Volume})$ can be informative over weekly horizons, especially when the direction is consistent and not driven by one-off events.

3. Congestion as a stress indicator

Mempool spikes can signal stress (panic settlement, exchange movements) or exuberance (mania). In both cases, it flags a regime where volatility and tail risk rise.

Failure modes

- On-chain data is noisy due to batching, exchange behavior, L2 usage, and non-economic transactions.
- Many on-chain metrics are more explanatory than predictive at short horizons.

5) Mining, Difficulty & Miner Balance Sheets (supply pressure and structural health)

Typical data

- Hash rate, difficulty
- Block interval statistics
- Miner revenue proxies: subsidy + fees, revenue per hash
- (If available) miner reserves / miner exchange outflows

Why it can predict

Mining data is not about “hash rate predicts price tomorrow.” The more realistic mechanism is **miner constraints and supply**.

1. Revenue per hash as profitability stress

A useful proxy:

$$\text{Rev}/\text{Hash}_t \approx \frac{(\text{Subsidy} + \text{Fees}) \cdot P_t}{\text{HashRate}_t}$$

When Rev/Hash compresses sharply, miners face balance-sheet stress and may liquidate BTC to cover OPEX—creating real sell pressure.

2. Difficulty adjustment lag creates temporary shocks

Difficulty adjusts with lag. If hash rate drops suddenly, miners temporarily earn more per hash until difficulty adjusts; if hash rate rises, revenue per hash compresses.

This lag can influence miner behavior over 1–3 week windows.

3. Hash rate as long-horizon confidence / capital commitment

Sustained hash rate growth implies long-term capital investment. It is more

informative as a structural indicator than as a short-term trading signal.

Failure modes

- Hash rate is typically exogenous to short-term price moves; treating it as a short-horizon predictor often disappoints.
 - Miner behavior is heterogeneous; without reserve/outflow data, inference can be indirect.
-

6) Macro Risk Regime (the “weather system” that decides which crypto signals work)

Typical data

- Equity indices (Nasdaq/S&P), VIX
- Rates (2Y/10Y), real yields (if available)
- USD strength (DXY), liquidity proxies

Why it can predict

Macro doesn't predict the next candle. It predicts **the regime**—and the regime determines the payoff of other signals.

1. BTC as a risk asset in many regimes

When liquidity tightens or risk aversion spikes, BTC behaves like a high-beta asset. Macro variables help identify periods where crypto-native signals (on-chain, flow, momentum) are likely to be dominated by broad deleveraging.

2. Regime filters reduce model error

A practical approach is to condition signals on macro state:

- use momentum only when volatility is moderate and risk is “on”
- down-weight on-chain signals during macro shock regimes

Failure modes

- BTC can decouple in crypto-specific narratives (e.g., idiosyncratic catalysts), so macro is not always dominant.
 - Macro effects are horizon-dependent and can appear with lags.
-

Putting It Together: What a "Good" Predictor Actually Looks Like

A strong BTC forecasting setup is usually not one model—it's a **stack**:

1. **Regime detector**: volatility + macro + liquidity → determines "trend vs chop vs stress"
2. **Directional module**: momentum + flows + derivatives crowding
3. **Risk module**: liquidation risk + liquidity → predicts tail probability and position size
4. **Slow fundamentals overlay**: on-chain activity + miner stress → medium-horizon bias

Why signals are conditionally predictive

Most BTC predictors fail because they assume stationarity. In reality, BTC changes regimes: leverage cycles, liquidity cycles, and macro cycles. The same indicator can flip sign across regimes (funding is the classic example).

Conclusion

The most useful BTC predictors fall into six sectors: **spot microstructure, derivatives positioning, flows/market access, on-chain usage, mining economics, and macro regime variables**. Their predictive value is not constant; it depends on horizon and regime. For modeling, the best practice is to (i) match each feature to a clear economic mechanism, (ii) test predictiveness by horizon, and (iii) explicitly include regime conditioning to avoid brittle signals.

If you want to make this "A+ obvious," I can add one extra page with:

- a small **feature-to-horizon matrix**, and
- a short **backtesting plan** (targets, metrics, and what to control for), so it looks like a real quant research memo rather than just a description.



